

Fluxion LMS 코드 컨벤션 (2025 Edition)

목적: Fluxion LMS 백엔드·프론트 모든 구성원이 동일한 현대(2025) 실무 기준으로 코드를 작성·리뷰·배포하도록 가이드한다.

목차

- 기본 규칙
- 프로젝트 구조
- 명명 규칙
- 의존성 규칙
- 코드 스타일
- 모듈 구조
- 테스트 규칙
- 보안 규칙
- 성능 최적화
- 문서화
- 필수 코딩 습관
- Git 워크플로우
- 로깅

기본 규칙

1. 개발 환경

- Node.js: 22.x LTS (최신 LTS 버전)
- NestJS: 11.x
- TypeScript: 5.x
- ESLint: 8.x
- Prettier: 3.x

2. 들여쓰기 & 공백

구분	규칙
들여쓰기	2 spaces (Prettier tabWidth: 2)
연산자·점표	앞뒤 1칸 공백
파일 마지막	\n 1줄
종괄호	여는 종괄호 뒤 줄바꿈

```
// Good
if (condition) {
  doSomething();
}

// Bad
if(condition){
doSomething();
}
```

3. 세미콜론

Prettier `semi: true` 로 자동 삽입. (여부가 아닌 일관성이 목표)

```
// Good
const greeting = 'hello';
const user = new User();

// Bad
const greeting = 'hello'
const user = new User()
```

4. 따옴표 & 템플릿

Prettier `singleQuote: true`

백틱 ` 허용 - 다중행 템플릿 필요 시 사용.

```
// Good
const name = 'John';
const message = `Hello ${name}`;

// Bad
const name = "John";
```

5. 코드 포매팅

- 최신 Prettier 3.x와 ESLint 8.x를 사용하여 일관된 코드 스타일 유지
- 최대 줄 길이는 120자로 제한
- 함수는 단일 책임 원칙을 따르고 최대 20줄을 넘지 않도록 권장

프로젝트 구조

1. 디렉토리 구조

```
src/
├── core/                # 핵심 인프라스트럭처
│   ├── config/         # 애플리케이션 설정
│   ├── security/       # 보안 관련
│   ├── cookie/         # 쿠키 관리
│   └── middleware/     # 핵심 미들웨어
├── shared/             # 공유 모듈
│   ├── dto/            # 데이터 전송 객체
│   ├── utils/          # 유틸리티 함수
│   ├── constants/      # 상수값
│   └── types/           # 공통 타입 정의
├── modules/            # 비즈니스 모듈
│   ├── auth/           # 인증 모듈
│   └── web/             # 웹 모듈
│       ├── api/         # API 모듈
│       ├── backoffice/  # 백오피스 모듈
│       └── portal/       # 포털 모듈
├── infrastructure/     # 외부 서비스 통합
│   ├── email/          # 이메일 서비스
│   ├── payment/        # 결제 서비스
│   └── fileupload/      # 파일 업로드
└── presentation/       # 프레젠테이션 레이어
    ├── handlebars/     # 템플릿 엔진
    └── swagger/         # API 문서화
```

1-2. 파일 구조

- 각 모듈은 다음 파일들을 포함
 - `*.module.ts` : 모듈 정의
 - `*.controller.ts` : 컨트롤러
 - `*.service.ts` : 서비스 로직
 - `*.repository.ts` : 데이터 접근 계층
 - `*.entity.ts` : 엔티티 정의

- *.dto.ts : DTO 클래스
- *.interface.ts : 인터페이스 정의
- *.spec.ts : 단위 테스트

1-2-1. 모듈별 파일 구성 예시

사용자 모듈 (User Module)

```
src/modules/web/portal/user/
├─ user.module.ts          # 모듈 정의
├─ user.controller.ts      # 컨트롤러
├─ user.service.ts        # 서비스 로직
├─ user.repository.ts      # 데이터 접근 계층
├─ user.entity.ts         # 엔티티 정의
├─ user.dto.ts            # DTO 클래스들
├─ user.interface.ts      # 인터페이스 정의
├─ user.spec.ts           # 단위 테스트
└─ user_history/          # 하위 기능 모듈
    ├─ user-history.module.ts
    ├─ user-history.controller.ts
    ├─ user-history.service.ts
    └─ user-history.entity.ts
```

홈 모듈 (Home Module)

```
src/modules/web/portal/home/
├─ home.module.ts          # 모듈 정의
├─ home.controller.ts      # 컨트롤러
├─ home.service.ts        # 서비스 로직
├─ home.dto.ts            # DTO 클래스들
├─ home.entity.ts         # 엔티티 정의
└─ home.spec.ts           # 단위 테스트
```

API 모듈 (API Module)

```
src/modules/web/api/
├─ api.module.ts          # API 모듈 정의
├─ users/                 # 사용자 API
│   ├─ user.controller.ts
│   ├─ user.interface.ts
│   ├─ user.module.ts
│   └─ user.service.ts
└─ courses/               # 강의 API
    ├─ course.controller.ts
    ├─ course.interface.ts
    ├─ course.module.ts
    └─ course.service.ts
```

백오피스 모듈 (Backoffice Module)

```
src/modules/web/backoffice/
├─ backoffice.module.ts    # 백오피스 모듈 정의
├─ dashboard/             # 대시보드
│   ├─ dashboard.controller.ts
│   ├─ dashboard.service.ts
│   └─ dashboard.module.ts
├─ user-management/       # 사용자 관리
│   ├─ user-management.controller.ts
│   ├─ user-management.service.ts
│   └─ user-management.module.ts
└─ settings/              # 설정 관리
    ├─ settings.controller.ts
    ├─ settings.service.ts
    └─ settings.module.ts
```

1-2-2. Views 폴더 구조

```

views/
├── index.hbs           # 메인 페이지
├── api/                # API 관련 뷰 (필요시)
│   └── docs.hbs
├── backoffice/        # 백오피스 관리자 뷰
│   ├── partials/      # 백오피스 공통 부분 템플릿
│   │   ├── header.hbs # 백오피스 헤더
│   │   ├── footer.hbs # 백오피스 푸터
│   │   ├── navigation.hbs # 백오피스 네비게이션
│   │   ├── sidebar.hbs # 백오피스 사이드바
│   │   ├── modal.hbs  # 백오피스 모달
│   │   └── components/ # 백오피스 재사용 컴포넌트
│   │       ├── button.hbs # 버튼 컴포넌트
│   │       ├── form.hbs  # 폼 컴포넌트
│   │       ├── table.hbs # 테이블 컴포넌트
│   │       └── pagination.hbs # 페이지네이션 컴포넌트
│   ├── dashboard.hbs  # 대시보드
│   ├── users.hbs      # 사용자 관리
│   ├── courses.hbs    # 강의 관리
│   └── settings.hbs   # 설정 관리
├── portal/            # 포털 사용자 뷰
│   ├── partials/      # 포털 공통 부분 템플릿
│   │   ├── header.hbs # 포털 헤더
│   │   ├── footer.hbs # 포털 푸터
│   │   ├── navigation.hbs # 포털 네비게이션
│   │   ├── sidebar.hbs # 포털 사이드바
│   │   ├── modal.hbs  # 포털 모달
│   │   └── components/ # 포털 재사용 컴포넌트
│   │       ├── button.hbs # 버튼 컴포넌트
│   │       ├── form.hbs  # 폼 컴포넌트
│   │       ├── card.hbs  # 카드 컴포넌트
│   │       └── pagination.hbs # 페이지네이션 컴포넌트
│   ├── home/
│   │   └── index.hbs   # 홈페이지
│   ├── user/
│   │   ├── login.hbs  # 로그인
│   │   ├── register.hbs # 회원가입
│   │   ├── profile.hbs # 프로필
│   │   └── myInfo.hbs  # 내 정보
│   └── board/
│       ├── list.hbs   # 게시판 목록
│       └── detail.hbs # 게시판 상세

```

1-2-3. 공통 모듈 파일 구성

```

src/shared/
├── dto/                # 공통 DTO
│   ├── auth.dto.ts
│   ├── login.dto.ts
│   └── page.dto.ts
├── utils/              # 유틸리티 함수
│   ├── common.util.ts
│   ├── date.util.ts
│   └── string.util.ts
├── constants/          # 상수값
│   ├── app.constants.ts
│   └── error.constants.ts
└── types/              # 공통 타입 정의
    ├── api.types.ts
    └── common.types.ts

```

1-2-4. 인프라스트럭처 모듈 파일 구성

```
src/infrastructure/
├─ email/           # 이메일 서비스
│  └─ email.module.ts
│  └─ email.service.ts
├─ payment/        # 결제 서비스
│  └─ payment.dto.ts
│  └─ payment.gateway.ts
│  └─ payment.module.ts
│  └─ payment.service.ts
└─ fileupload/     # 파일 업로드
   └─ fileupload.controller.ts
   └─ fileupload.module.ts
   └─ fileupload.service.ts
```

명명 규칙

1. 파일명

- 모두 소문자로 작성
- 단어 사이는 하이픈(-)으로 구분
- 용도별 접미사 추가

```
user.controller.ts
user.service.ts
user.module.ts
user.entity.ts
user.dto.ts
user.interface.ts
```

2. 클래스명

- PascalCase 사용
- 역할에 따른 접미사 추가

```
export class UserController {}
export class UserService {}
export class CreateUserDto {}
export class UserEntity {}
```

3. 변수/메서드명

- camelCase 사용
- 불리언은 is, has, can 등의 접두사 사용
- private 멤버는 언더스코어(_) 접두사 사용
- 메서드명은 동사로 시작

```
const userName = 'John';
const isActive = true;
const hasPermission = false;
private _secretKey: string;

async findUserById(id: number): Promise<User> {}
async createNewUser(user: CreateUserDto): Promise<User> {}
```

4. 상수

- 대문자와 언더스코어(_) 사용

```
export const MAX_CONNECTIONS = 100;
export const API_BASE_URL = 'https://api.example.com';
```

5. 인터페이스

- 접두사 'I' 사용하지 않음

```
// Good
interface User {
  id: number;
  name: string;
}

// Bad
interface IUser {
  id: number;
  name: string;
}
```

의존성 규칙

1. 계층별 의존성 방향

```
core ← shared ← modules
    ↑         ↑
    └────────┘
infrastructure
  ↑
presentation
```

2. 의존성 주입

- 생성자 기반 의존성 주입 사용
- `private readonly`로 선언

```
// Good
@Injectable()
export class UserService {
  constructor(
    private readonly userRepository: UserRepository,
    private readonly mailService: MailService
  ) {}
}

// Bad
@Injectable()
export class UserService {
  @Inject()
  private userRepository: UserRepository;
}
```

3. 순환 의존성 방지

- 모듈 간 순환 참조 금지
- 필요한 경우 shared 모듈을 통해 공통 기능 제공

코드 스타일

1. 비동기 처리

```
// Good
async function getUser(id: number): Promise<User> {
  const user = await this.userRepository.findOne(id);
  return user;
}

// Bad
function getUser(id: number): Promise<User> {
  return this.userRepository.findOne(id)
    .then(user => user);
}
```

2. 예외 처리

서비스 계층 `try/catch` → 의도적 예외 변환만.

전역 `AllExceptionsFilter` 에서 로깅·HTTP 매핑.

```
// Good
async create(dto: CreateUserDto): Promise<User> {
  try {
    const user = await this.userRepository.create(dto);
    await this.eventEmitter.emit('user.created', user);
    return user;
  } catch (error) {
    this.logger.error('Failed to create user', error);
    throw new InternalServerErrorException('Failed to create user');
  }
}
```

3. 타입 안정성

`any` 금지, 필요시 `unknown` → 좁히기.

`strictNullChecks`, `exactOptionalPropertyTypes` 활성화.

```
// Good
function processUser(user: User): void {}

// Bad
function processUser(user: any) {}
```

4. TypeScript 5.x 최신 기능 활용

```
// const assertions 활용
const config = {
  database: {
    host: 'localhost',
    port: 5432
  }
} as const;

// satisfies 연산자 활용
const userRoles = ['admin', 'user', 'guest'] satisfies string[];

// 타입 안전한 키 접근
type UserRole = typeof userRoles[number]; // 'admin' | 'user' | 'guest'

// 향상된 타입 추론
const errorCodes = {
  VALIDATION_ERROR: 'VALIDATION_ERROR',
  NOT_FOUND: 'NOT_FOUND',
  UNAUTHORIZED: 'UNAUTHORIZED'
} satisfies Record<string, string>;
```

모듈 구조

1. 모듈 구성

- 기능별로 모듈 분리
- 모듈은 독립적으로 동작

```
@Module({
  imports: [
    DatabaseModule,
    RedisModule,
    BullModule.registerQueue({ name: 'email' }),
  ],
  providers: [UserRepoPrisma, UserService],
  controllers: [UserController],
})
export class UserModule {}
```

2. 컨트롤러 구조

- 라우트 처리만 담당
- 비즈니스 로직은 서비스로 위임
- 권장: 복잡한 로직에 try-catch 적용
- 선택적: 필요에 따라 RBAC 권한 시스템 적용 (레벨 1-3 참고)

```
// 레벨 3 예시: 관리자 전용 API
@Controller('admin/users')
@UseGuards(JwtAuthGuard, RBACGuard) // JWT + RBAC 가드 적용
@SetResource('user') // 리소스 설정
export class AdminUserController {
  constructor(
    private readonly userService: UserService,
    private readonly errorLogger: ErrorLoggerService // 에러 로깅 (권장)
  ) {}

  @Get()
  @RequirePermission('read') // 모든 사용자 조회 권한 필요
  async getAllUsers(): Promise<User[]> {
    try {
      return await this.userService.findAll();
    } catch (error) {
      this.errorLogger.error('AdminUserController', '전체 사용자 목록 조회 실패', error);
      throw new HttpException('사용자 목록 조회에 실패했습니다.', HttpStatus.INTERNAL_SERVER_ERROR);
    }
  }

  @Delete('/:id')
  @RequirePermission('delete') // 사용자 삭제 권한 필요
  async deleteUser(@Param('id') id: string): Promise<void> {
    try {
      await this.userService.delete(+id);
    } catch (error) {
      this.errorLogger.error('AdminUserController', '사용자 삭제 실패', error, { id });
      throw new HttpException('사용자 삭제에 실패했습니다.', HttpStatus.INTERNAL_SERVER_ERROR);
    }
  }
}
```

2-1. 에러 처리 규칙

- 복잡한 비즈니스 로직은 try-catch 권장
- 에러 발생 시 로그 기록 (성공 시는 중요 이벤트만 기록)
- 사용자 친화적 에러 메시지 제공
- 적절한 HTTP 상태 코드 설정

2-2. RBAC 권한 시스템 적용

- 선택적 적용: 공개 API는 적용하지 않음 (레벨 1)
- JWT 인증만: 본인 데이터 접근 시 (레벨 2)

- **JWT + RBAC**: 관리자 기능이나 타인 데이터 접근 시 (레벨 3)
- 권한 종류: `read`, `write`, `delete`, `approve`, `admin`

2-3. 의존성 주입 규칙

```
constructor(
  private readonly userService: UserService,      // 비즈니스 로직 서비스
  private readonly errorLogger: ErrorLoggerService // 에러 로깅 (권장)
) {}
```

3. 모듈별 폴더 분리 관리 방식

3-1. 기본 원칙

- 모든 기능을 폴더로 분리: 메인 컨트롤러 없이 기능별로 폴더 구성
- 명확한 책임 분리: 각 폴더는 특정 기능만 담당
- 확장성: 새로운 기능은 새로운 폴더로 추가
- 유사한 기능은 하나의 폴더에 포함: 유사한 기능은 endpoint기준 15개까지 하나의 폴더에 포함

3-2. 폴더 구조 예시

```
src/modules/web/portal/user/
├─ user.module.ts      # 모듈 정의
├─ profile/            # 프로필 관리
│  └─ profile.controller.ts
│  └─ profile.service.ts
│  └─ profile.repository.ts
│  └─ profile.entity.ts
│  └─ profile.dto.ts
├─ auth/               # 인증 관련
│  └─ auth.controller.ts
│  └─ auth.service.ts
│  └─ auth.repository.ts
│  └─ auth.dto.ts
├─ history/            # 히스토리
│  └─ history.controller.ts
│  └─ history.service.ts
│  └─ history.repository.ts
│  └─ history.entity.ts
└─ settings/           # 설정
   └─ settings.controller.ts
   └─ settings.service.ts
   └─ settings.repository.ts
   └─ settings.dto.ts
```

3-3. 라우팅 규칙

```
// 프로필 관리
@Controller('users/profile')
export class ProfileController {
  @Get() // GET /users/profile
  @Put() // PUT /users/profile
  @Post('avatar') // POST /users/profile/avatar
}

// 인증 관련
@Controller('users/auth')
export class AuthController {
  @Post('login') // POST /users/auth/login
  @Post('logout') // POST /users/auth/logout
  @Post('refresh') // POST /users/auth/refresh
}

// 히스토리
@Controller('users/history')
export class HistoryController {
  @Get() // GET /users/history
  @Get(':id') // GET /users/history/:id
}

// 설정
@Controller('users/settings')
export class SettingsController {
  @Get() // GET /users/settings
  @Put() // PUT /users/settings
  @Post('notifications') // POST /users/settings/notifications
}
```

3-4. 모듈 구성

```
@Module({
  imports: [DatabaseModule],
  controllers: [
    ProfileController,
    AuthController,
    HistoryController,
    SettingsController,
  ],
  providers: [
    ProfileService,
    AuthService,
    HistoryService,
    SettingsService,
  ],
})
export class UserModule {}
```

3-5. 장점

- 명확한 책임 분리: 각 폴더가 하나의 기능만 담당
- 확장성: 새로운 기능 추가 시 새로운 폴더 생성
- 유지보수성: 관련 코드가 한 곳에 집중
- 팀 협업: 여러 개발자가 동시에 다른 기능 작업 가능
- 테스트 용이성: 각 기능별로 독립적인 테스트 작성 가능

테스트 규칙

레이어	툴	커버리지 목표
Domain / App	Jest(tests + vitest)	≥ 80 %
Interfaces e2e	@nestjs/testing + Supertest	Δ -2 % 이하

2. 테스트 구조

- Describe-Context-It 패턴 사용

```
describe('UserService', () => {
  let service: UserService;

  beforeEach(async () => {
    const module = await Test.createTestingModule({
      providers: [UserService],
    }).compile();

    service = module.get<UserService>(UserService);
  });

  it('should be defined', () => {
    expect(service).toBeDefined();
  });
});
```

보안 규칙

1. RBAC (Role-Based Access Control) 권한 시스템

- 선택적 적용: 필요에 따라 3가지 레벨로 적용
- 페이지별 세밀한 권한 제어
- 계층적 권한 구조 (관리자 > 승인자 > 편집자 > 작성자 > 독자)

1-0. RBAC 적용 레벨

레벨 1: 공개 API (인증/권한 불필요)

```
@Controller('public')
export class PublicController {
  @Get('posts') // 누구나 접근 가능
  async getPosts() { ... }

  @Get('health') // 헬스체크
  async healthCheck() { ... }
}
```

레벨 2: 로그인 필요 (JWT 인증만)

```
@Controller('users')
@UseGuards(JwtAuthGuard) // JWT 인증만
export class UserController {
  @Get('my-profile') // 본인 정보만 조회
  async getMyProfile(@Req() req) {
    return this.userService.findById(req.user.id);
  }
}
```

레벨 3: 권한 관리 필요 (JWT + RBAC)

```
@Controller('admin/users')
@UseGuards(JwtAuthGuard, RBACGuard) // 인증 + 권한 둘 다
@SetResource('user')
export class AdminUserController {
  @Get()
  @RequirePermission('read') // 다른 사용자 정보 조회 권한
  async getAllUsers() { ... }

  @Delete(':id')
  @RequirePermission('delete') // 사용자 삭제 권한
  async deleteUser() { ... }
}
```

레벨 4: 하이브리드 적용 (메서드별 선택적 보안)

```

@Controller('posts')
export class PostController {
  @Get() // 공개 - 권한 없음
  async getPosts() { ... }

  @Get('/:id') // 공개 - 권한 없음
  async getPost() { ... }

  @Post() // 로그인 필요
  @UseGuards(JwtAuthGuard)
  async createPost(@Req() req) { ... }

  @Delete('/:id') // 권한 필요
  @UseGuards(JwtAuthGuard, RBACGuard)
  @SetResource('post')
  @RequirePermission('delete')
  async deletePost() { ... }
}

```

1-1. 폴더별 역할 분담

```

/src/core/security/auth/ # 공통 라이브러리 (RBAC 핵심 로직)
├─ rbac.service.ts       # 권한 확인, 사용자 권한 조회, 캐싱
├─ jwt-auth.service.ts   # JWT 토큰 처리
└─ user-auth.service.ts  # 사용자 인증

/src/modules/auth/       # 비즈니스 로직 (애플리케이션별 처리)
├─ rbac.guard.ts         # 요청별 권한 검증
├─ permission.decorator.ts # 권한 데코레이터
└─ auth.service.ts       # 인증 서비스

```

1-2. RBAC 적용 방법

```

// 컨트롤러 레벨 적용
@Controller('users')
@UseGuards(RBACGuard) // RBAC 가드 적용
@SetResource('user') // 리소스 설정
export class UserController {
  constructor(
    private readonly userService: UserService,
    private readonly errorLogger: ErrorLoggerService // 권장
  ) {}

  @Get()
  @RequirePermission('read') // 읽기 권한 필요
  async findAll() { ... }

  @Post()
  @RequirePermission('write') // 쓰기 권한 필요
  async create() { ... }

  @Delete('/:id')
  @RequirePermission('delete') // 삭제 권한 필요
  async delete() { ... }

  @Put('/:id/approve')
  @RequirePermission('approve') // 승인 권한 필요
  async approve() { ... }
}

```

1-3. 권한 종류 정의

- 기본 권한: `read`, `write`, `delete`, `approve`, `admin`
- 확장 권한: `read_others`, `write_others`, `delete_others`, `bulk_operation`, `export`
- 조건부 권한: 부서별, 시간별, IP별 권한 제어

1-4. 기본 데코레이터

```
// 기본 권한 데코레이터
@RequirePermission('read')
@RequirePermission('write')
@RequirePermission('delete')
@RequirePermission('admin')

// 리소스 설정
@SetResource('user')
@SetResource('board')
```

2. 인증/인가 (기존)

- JWT 토큰은 HttpOnly 쿠키로 관리
- RBAC 시스템과 연동

```
@UseGuards(JwtAuthGuard, RBACGuard) // JWT 인증 + RBAC 권한 확인
@SetResource('admin')
@RequirePermission('admin')
@Get('admin')
async adminOnly() {
    return this.adminService.getData();
}
```

3. 데이터 검증

- class-validator 사용
- DTO에서 유효성 검사 규칙 정의

```
export class CreateUserDto {
    @IsString()
    @MinLength(2)
    readonly name: string;

    @IsEmail()
    readonly email: string;
}
```

4. 공통 헤더

Helmet, CORS, RateLimiter(Redis) 전역.

5. 보안 업데이트 (OWASP 2024)

```
// 1. 인젝션 방지
@Injectable()
export class UserService {
  async findUsers(searchTerm: string): Promise<User[]> {
    // Prisma를 사용하여 SQL 인젝션 방지
    return this.prisma.user.findMany({
      where: {
        name: {
          contains: searchTerm, // 자동으로 이스케이프됨
        },
      },
    });
  }
}

// 2. 인증 우회 방지
@UseGuards(JwtAuthGuard)
@Controller('users')
export class UserController {
  @Get('profile')
  async getProfile(@Request() req) {
    // JWT에서 사용자 정보 추출
    return this.userService.findById(req.user.id);
  }
}

// 3. 민감 데이터 로그 방지
@Injectable()
export class LoggingService {
  private sanitizeForLog(data: any): any {
    const sensitiveFields = ['password', 'token', 'ssn', 'cardNumber'];
    return Object.keys(data).reduce((acc, key) => {
      acc[key] = sensitiveFields.includes(key) ? '[REDACTED]' : data[key];
      return acc;
    }, {});
  }
}
```

성능 최적화

1. DB

Prisma `select/include` 로 N+1 방지.

인덱스 → 설계 문서에 기록 (`schema.prisma` `@@index`).

```
// Good - N+1 방지
const users = await prisma.user.findMany({
  include: {
    profile: true,
    posts: {
      select: {
        id: true,
        title: true,
      },
    },
  },
});

// Bad - N+1 문제 발생
const users = await prisma.user.findMany();
for (const user of users) {
  const profile = await prisma.profile.findUnique({
    where: { userId: user.id },
  });
}
```

2. 캐싱

- 자주 사용되는 데이터 캐싱

- 캐시 무효화 전략 수립

```
@UseInterceptors(CacheInterceptor)
@CacheTTL(60)
@GetMapping('featured')
findFeatured() {
    return this.productService.getFeatured();
}
```

문서화

1. API 문서화

- Swagger 사용
- 모든 API 엔드포인트에 대한 설명 추가

```
@ApiOperation({ summary: '사용자 목록 조회' })
@ApiResponse({ status: 200, description: '사용자 목록 반환' })
@GetMapping()
async findAll(): Promise<User[]> {
    return this.userService.findAll();
}
```

필수 코딩 습관

1. 에러 처리 및 로깅 규칙

- 중요한 비즈니스 로직은 try-catch 권장
- 에러 발생 시 로그 기록 (성공 시는 중요 이벤트만 선택적 기록)
- ErrorLoggerService 사용 권장

1-1. 컨트롤러 에러 처리 패턴

```
@Controller('users')
@UseGuards(RBACGuard)
@SetResource('user')
export class UserController {
    constructor(
        private readonly userService: UserService,
        private readonly errorLogger: ErrorLoggerService // 권장 의존성
    ) {}

    @Post()
    @RequirePermission('write')
    async create(@Body() createUserDto: CreateUserDto): Promise<User> {
        try {
            return await this.userService.create(createUserDto);
        } catch (error) {
            // 에러 발생 시 로그 기록
            this.errorLogger.error('UserController', '사용자 생성 실패', error, { userData: createUserDto });
            throw new HttpException('사용자 생성에 실패했습니다.', HttpStatus.INTERNAL_SERVER_ERROR);
        }
    }
}
```

1-2. 서비스 에러 처리 패턴

```

@Injectable()
export class UserService {
  constructor(
    private readonly userRepository: UserRepository,
    private readonly errorLogger: ErrorLoggerService
  ) {}

  async createUser(dto: CreateUserDto): Promise<User> {
    try {
      const user = await this.userRepository.create(dto);
      await this.eventEmitter.emit('user.created', user);
      return user;
    } catch (error) {
      this.errorLogger.error('UserService', '사용자 생성 실패', error, { dto });
      throw error; // 컨트롤러에서 처리하도록 재던지기
    }
  }
}

```

1-3. 에러 로깅 규칙

- 에러 로그 필수: 문제 해결을 위해 에러 발생 시 반드시 기록
- 성공 로그 선택적: 중요한 비즈니스 이벤트는 성공 시도 기록
- 컨텍스트 정보 포함: 에러 발생 시 관련 데이터 함께 기록
- 사용자 친화적 메시지: 클라이언트에는 기술적 세부사항 노출 금지

2. 로깅 표준화

- 로그 레벨 적절히 사용

```

// 에러 로깅
this.logger.error('Database connection failed', error.stack, 'DatabaseService');

// 중요 비즈니스 이벤트
this.logger.log('Payment processed successfully', 'PaymentService');

// 디버그 정보
this.logger.debug('Processing user request', { userId, action });

```

3. 트랜잭션 관리

- 데이터 정합성이 중요한 작업은 반드시 트랜잭션 사용

```

async transfer(from: string, to: string, amount: number): Promise<void> {
  await this.dataSource.transaction(async manager => {
    await manager.decrement(Account, { id: from }, 'balance', amount);
    await manager.increment(Account, { id: to }, 'balance', amount);
  });
}

```

4. 환경 설정 관리

- 환경변수는 ConfigService를 통해 접근
- 환경별 설정 파일 분리

```

@Injectable()
export class AppService {
  constructor(private configService: ConfigService) {
    const dbHost = this.configService.get<string>('database.host');
    const isProd = this.configService.get<boolean>('isProduction');
  }
}

```

5. 메모리 관리

- 대용량 데이터 처리시 스트림 사용
- 메모리 누수 방지

```
@Get('download')
async downloadLargeFile(@Res() response: Response) {
    const fileStream = createReadStream(filePath);
    fileStream.pipe(response);
}
```

6. 비동기 작업 최적화

- Promise.all 활용하여 병렬 처리

```
async getFullUserInfo(userId: string): Promise<UserInfo> {
    const [user, orders, preferences] = await Promise.all([
        this.userService.findOne(userId),
        this.orderService.findByUser(userId),
        this.preferenceService.findByUser(userId)
    ]);
    return { user, orders, preferences };
}
```

7. 타입 안정성

- 엄격한 타입 검사 설정 사용

```
{
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "strictFunctionTypes": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "exactOptionalPropertyTypes": true,
    "useUnknownInCatchVariables": true
  }
}
```

8. 순환 참조 방지

- forwardRef 사용은 최소화
- 모듈 간 의존성 그래프 관리

```
@Injectable()
export class UserService {
    constructor(
        @Inject(forwardRef(() => AuthService))
        private authService: AuthService
    ) {}
}
```

9. 이벤트 기반 아키텍처

- 느슨한 결합을 위한 이벤트 사용

```

@Injectable()
export class UserService {
  constructor(private emitter: EventEmitter2) {}

  async createUser(dto: CreateUserDto): Promise<User> {
    const user = await this.userRepository.create(dto);
    await this.emitter.emit('user.created', user);
    return user;
  }
}

@Injectable()
export class NotificationService {
  @OnEvent('user.created')
  async handleUserCreated(user: User) {
    await this.sendWelcomeEmail(user);
  }
}

```

10. API 버전 관리

- API 버전 명시적 관리

```

@Controller({
  version: '1',
  path: 'users'
})
export class UserController {
  @Get()
  @ApiVersion('1')
  findAll(): Promise<User[]> {
    return this.userService.findAll();
  }

  @Get()
  @ApiVersion('2')
  findAllV2(): Promise<UserDto[]> {
    return this.userService.findAllWithDetails();
  }
}

```

11. 캐시 전략

- 캐시 키 명명 규칙 준수
- 적절한 TTL 설정

```

@Injectable()
export class ProductService {
  @Cacheable('products:featured', { ttl: 3600 })
  async getFeaturedProducts(): Promise<Product[]> {
    return this.productRepository.findFeatured();
  }

  @CacheEvict('products:featured')
  async updateProduct(id: number, dto: UpdateProductDto): Promise<void> {
    await this.productRepository.update(id, dto);
  }
}

```

12. 데이터베이스 쿼리 최적화

- QueryBuilder 활용
- 적절한 인덱스 설정

```

async findActiveUsers(): Promise<User[]> {
  return this.userRepository
    .createQueryBuilder('user')
    .leftJoinAndSelect('user.profile', 'profile')
    .where('user.isActive = :isActive', { isActive: true })
    .orderBy('user.createdAt', 'DESC')
    .cache(true)
    .getMany();
}

```

13. 보안 베스트 프랙티스

- 입력 데이터 검증
- XSS, CSRF 방어

```

@Post()
@UsePipes(new ValidationPipe({ transform: true }))
@UseGuards(CsrfGuard)
async createUser(@Body() dto: CreateUserDto): Promise<User> {
  dto.email = this.sanitizer.sanitize(dto.email);
  return this.userService.create(dto);
}

```

14. 헬스체크 구현

- 주요 서비스 상태 모니터링

```

@Injectable()
export class HealthService {
  @HealthCheck()
  check() {
    return {
      database: this.checkDatabase(),
      redis: this.checkRedis(),
      externalApis: this.checkExternalApis()
    };
  }
}

```

15. 우아한 종료 처리

- 서버 종료 시 리소스 정리

```

async onApplicationShutdown(signal: string) {
  this.logger.log(`Shutdown signal received: ${signal}`);
  await this.closeDbConnections();
  await this.cancelOngoingTasks();
  await this.closeRedisConnections();
}

```

16. AI 코드 어시스턴트 활용

- GitHub Copilot, Cursor AI 등을 활용한 개발 효율성 향상
- AI 생성 코드는 리뷰 후 사용 권장
- 보안에 민감한 코드는 신중하게 검토

AI 도구 사용 가이드라인:

```

// Good - AI 생성 후 리뷰된 코드
async validateUser(user: User): Promise<boolean> {
  if (!user.email || !user.name) {
    return false;
  }
  return this.emailService.isValidEmail(user.email);
}

```

권장사항:

- 핵심 비즈니스 로직은 직접 작성 권장
- AI 생성 코드도 적절한 테스트 작성
- 코드 리뷰 시 품질 검증

Git 워크플로우

1. 브랜치 전략

```
main          # 운영 배포 브랜치 (항상 배포 가능한 상태 유지)
├─ develop    # 개발 통합 브랜치 (다음 릴리즈 준비)
├─ feature/*   # 기능 개발 브랜치 (feature/user-auth)
├─ hotfix/*    # 긴급 수정 브랜치 (hotfix/login-bug)
└─ release/*   # 릴리즈 준비 브랜치 (release/v1.2.0)
```

2. 커밋 메시지 컨벤션

Conventional Commits 규칙을 따릅니다.

형식: `type(scope): description`

타입:

- `feat`: 새로운 기능
- `fix`: 버그 수정
- `docs`: 문서 변경
- `style`: 코드 포매팅
- `refactor`: 코드 리팩토링
- `test`: 테스트 추가/수정
- `chore`: 빌드 설정 등

예시:

```
feat(auth): JWT 인증 시스템 구현
fix(user): 사용자 등록 시 이메일 중복 검사 오류 수정
docs(api): 사용자 API 문서 업데이트
test(payment): 결제 서비스 단위 테스트 추가
refactor(database): 사용자 쿼리 성능 최적화
```

3. PR/MR 규칙

- 제목: 커밋 메시지 형식과 동일
- 설명: 변경 내용, 테스트 방법, 스크린샷(UI 변경 시)
- 최소 1명 이상의 코드 리뷰 필수
- CI/CD 체크 통과 후 머지

4. 코드 리뷰 체크리스트

기능성

- [] 요구사항을 정확히 구현했는가?
- [] 에러 처리가 적절한가?
- [] 성능 이슈는 없는가?

보안

- [] 입력 검증이 적절한가?
- [] 인증/인가가 올바르게 구현되었는가?
- [] 민감한 데이터가 로그에 노출되지 않는가?

코드 품질

- [] 변수/함수명이 명확한가?
- [] 중복 코드가 있는가?
- [] 테스트가 충분한가?

5. 자동화된 코드 품질 체크

```
# .github/workflows/code-quality.yml
name: Code Quality

on: [push, pull_request]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'

      - name: Install dependencies
        run: npm ci

      - name: Run linting
        run: npm run lint

      - name: Run type checking
        run: npm run type-check

      - name: Run tests
        run: npm run test:cov

      - name: SonarCloud Scan
        uses: SonarSource/sonarcloud-github-action@master
```

로깅

1. 구조화된 로깅

Winston 또는 Pino를 사용하여 구조화된 로그를 작성합니다.

```
// Good - 구조화된 로깅
this.logger.log({
  event: 'user.login.success',
  userId: user.id,
  timestamp: new Date().toISOString(),
  metadata: {
    ip: req.ip,
    userAgent: req.headers['user-agent'],
    sessionId: req.sessionID
  }
});

// Bad - 단순 문자열 로깅
this.logger.log(`User ${user.id} logged in`);
```

2. 로그 레벨 가이드

```

// ERROR: 시스템 오류, 즉시 대응 필요
this.logger.error('Database connection failed', {
  error: error.message,
  stack: error.stack,
  context: 'DatabaseService'
});

// WARN: 잠재적 문제, 모니터링 필요
this.logger.warn('High memory usage detected', {
  memoryUsage: process.memoryUsage(),
  threshold: '80%'
});

// INFO: 중요한 비즈니스 이벤트
this.logger.info('Payment processed successfully', {
  orderId: order.id,
  amount: order.amount,
  paymentMethod: order.paymentMethod
});

// DEBUG: 개발/디버깅 정보 (운영에서는 비활성화)
this.logger.debug('Processing user request', {
  userId,
  action,
  requestId: req.id
});

```

3. 파일 내부 구조

```

// 1. 임포트
import { Injectable } from '@nestjs/common';
import { User } from './user.entity';

// 2. 데코레이터
@Injectable()

// 3. 클래스 선언
export class UserService {
  // 4. 속성 선언
  private readonly users: User[];

  // 5. 생성자
  constructor(private readonly userRepository: UserRepository) {}

  // 6. 공개 메서드
  async findAll(): Promise<User[]> {
    return this.users;
  }

  // 7. 비공개 메서드
  private validateUser(user: User): boolean {
    return user.isActive;
  }
}

```